

Data Structures

Lecture 8

Trees

Abstract Data Type

References:

Data Structures and Algorithms in Java

Michael T. Goodrich and Roberto Tamassia.



University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar

Information Technology Research and
Development Center ITRDC

Email:

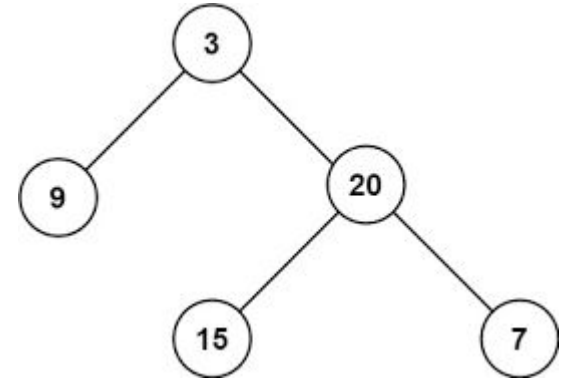
munqith.alattar@uokufa.edu.iq

Website:

<http://staff.uokufa.edu.iq/profile.html?munqith.alattar>

Topics

- Trees
- Binary tree
 - Binary expression tree
 - Array-Based Representation of a Binary Tree
 - Traversal of a Binary Tree
- Binary search tree,
- Heap tree

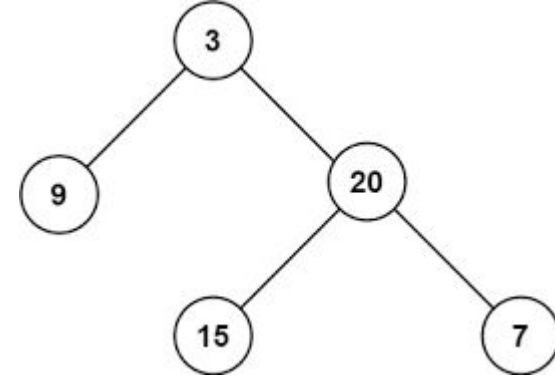


Data Structures

Tree

A tree stores elements *hierarchically*.

- A **tree** is usually visualized by drawing the connections between parents and children with straight lines.
- The top element is called the **root** of the tree.
- Two nodes that are children of the same parent are **siblings**.
- Each element (except the root) has a parent element and zero or more children elements.
- A node is **external** if it has no children, it is also known as **leaves**.
- A node is **internal** if it has one or more children.
- An **edge** of tree is a pair of nodes (u,v) such that u is the parent of v , or vice versa.
For example: $(9,3)$
- A **path** of tree is a sequence of nodes such that any two consecutive nodes in the sequence form an edge. For example: $(3,20,15)$



Data Structures

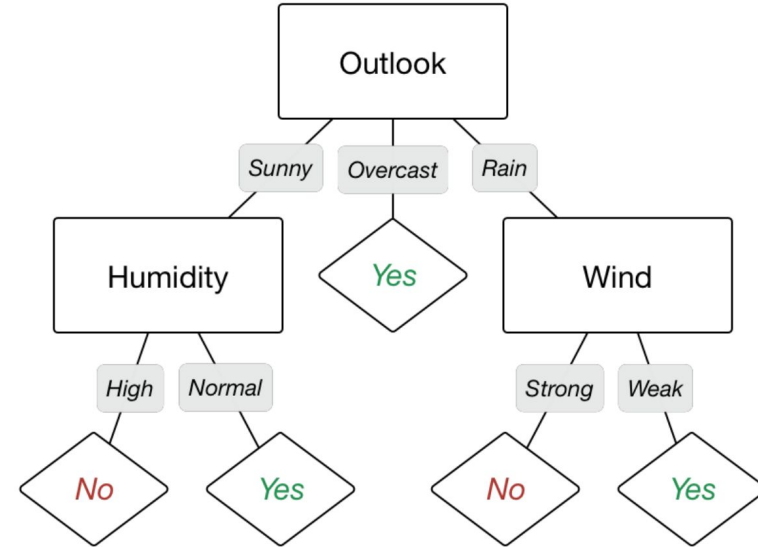
Binary Tree

A **binary tree** is a tree with the following properties:

- Every node has at most two children.
- Each child node is labeled as being either a left child or a right child.
- A left child precedes a right child in the order of children of a node.

The subtree rooted at a left or right child of a node is called a **left subtree** or **right subtree**, respectively.

A binary tree is **Proper Binary Tree (Full Binary Tree)** if each node has either zero or two children (every internal node has exactly two children).



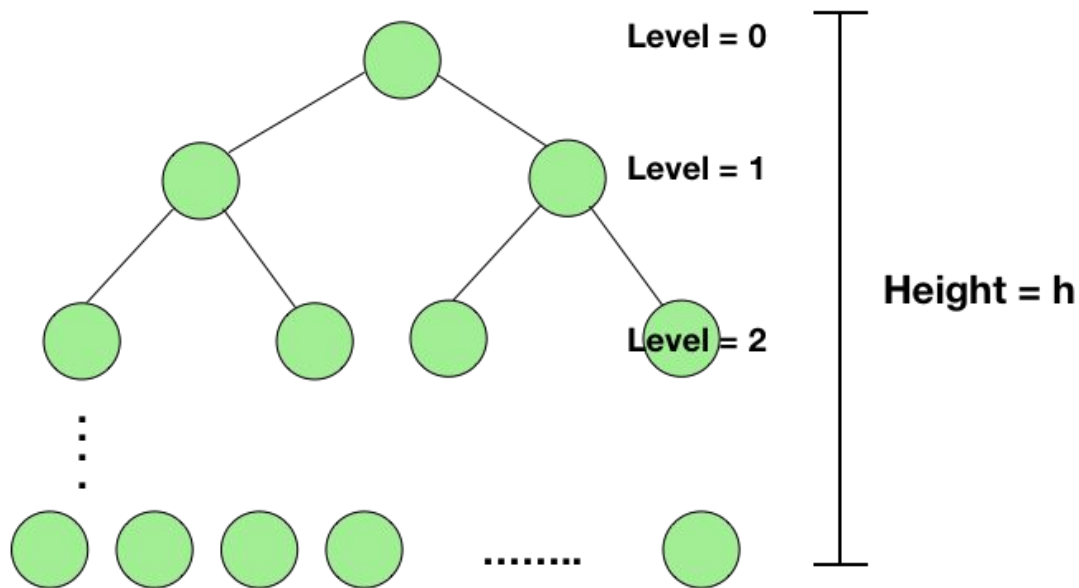
Data Structures

Binary Tree

Perfect binary tree, each leaf is at the same level and the and all the interior nodes have two children. It means that a perfect binary tree of height **h** has exactly **$2^{h+1}-1$** nodes.

For example

- Number of nodes at level 0 = $2^{0+1} - 1 = 1$
- Number of nodes at level 1 = $2^{1+1} - 1 = 3$
- Number of nodes at level 2 = $2^{2+1} - 1 = 7$
- ...



Data Structures

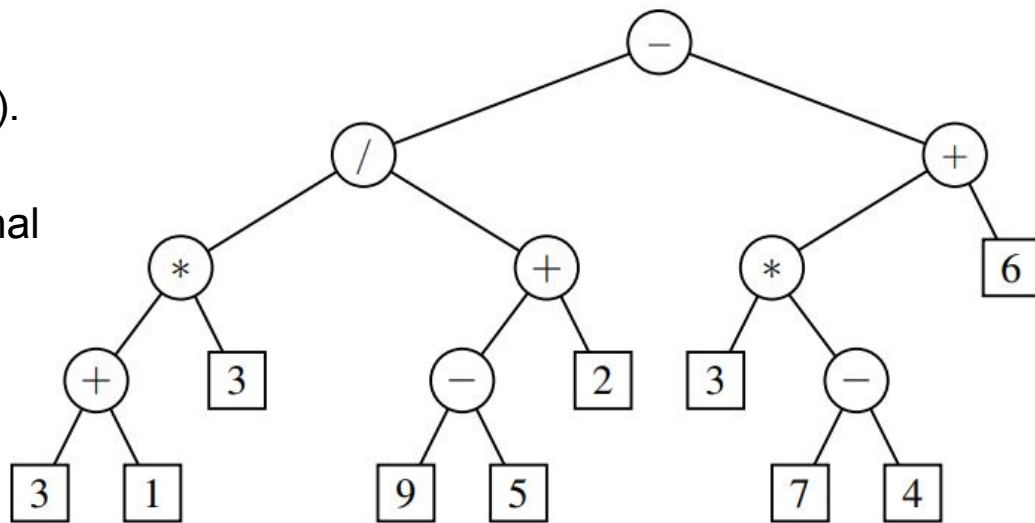
Binary expression tree

An arithmetic expression can be represented by a binary tree:

- leaves are associated with values,
- internal nodes are associated with one of the operators +, −, *, and /.

This tree represents the expression
 $((((3+1) * 3)/((9-5)+2))-((3 * (7-4))+6))$.

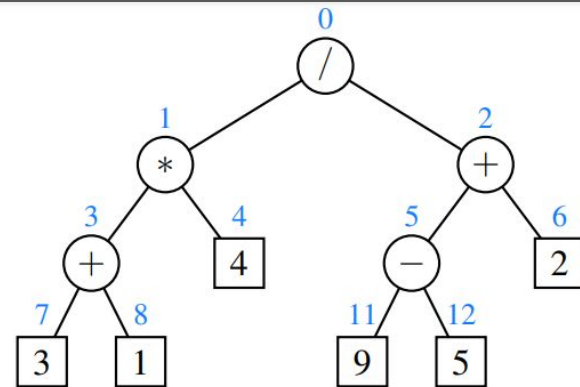
So, the value associated with the internal node labeled “/” is 2.



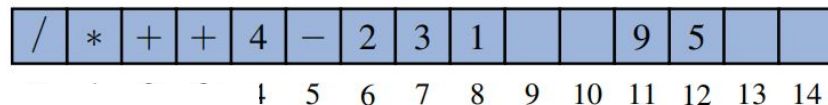
Data Structures

Array-Based Representation of a Binary Tree

A binary tree can be stored in a single array. By **numbering** the nodes from top to bottom, and from left to right for the nodes at the same level. Then the values at the nodes are stored in an array with the index corresponding to the numbers given to the nodes.



For every position p of T , let $f(p)$ be the integer defined as follows.



- If p is the root of T , then $f(p) = 0$.
- If p is the left child of position q , then $f(p) = 2f(q) + 1$.
- If p is the right child of position q , then $f(p) = 2f(q) + 2$.

Data Structures

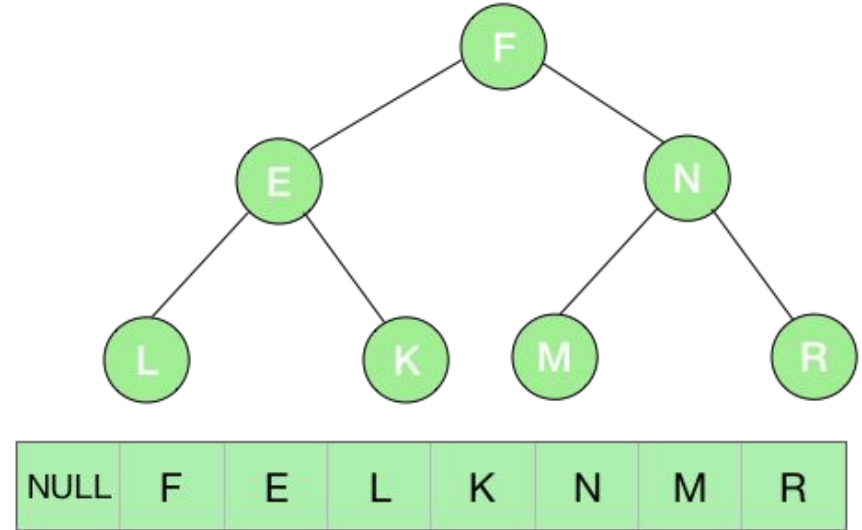
Traversal of a Binary Tree

A traversal of a tree T is accessing all the positions of T .

The three basic types of traversals are:

Preorder: the root of the tree is visited first and then the left subtree and finally the right subtree

```
function preorder(Node n):  
{  
    if(n != null)  
    {  
        print(n.data)  
        preorder(n.left)  
        preorder(n.right)  
    }  
}
```

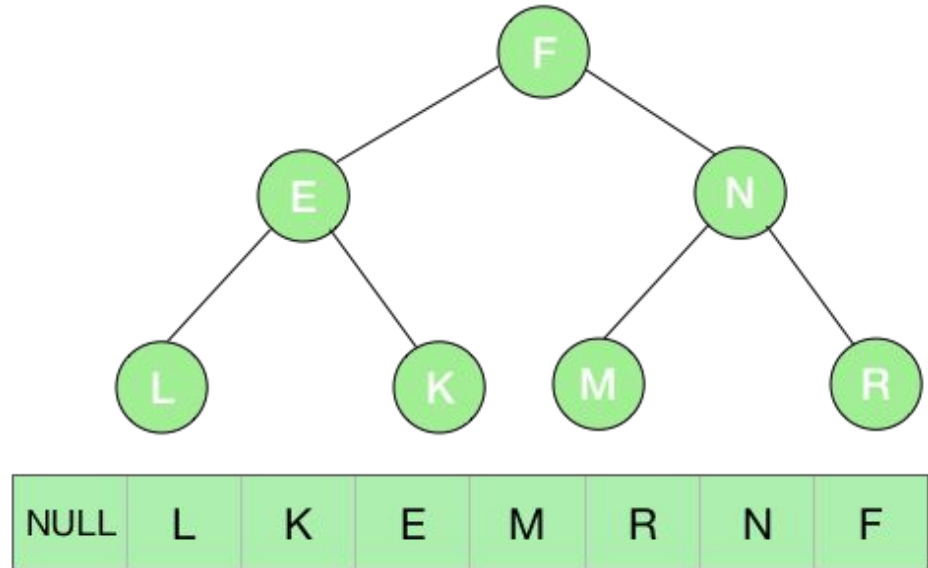


Data Structures

Traversal of a Binary Tree

Postorder: the left subtree is visited first, then the right subtree and lastly the root.

```
function postorder(Node n)
{
    if(n != null)
    {
        postorder(n.left)
        postorder(n.right)
        print(n.data)
    }
}
```



Data Structures

Traversal of a Binary Tree

Breadth-First Tree: visit all the positions at depth d before we visit the positions at depth $d + 1$. Since we are not traversing entire subtrees at once, we use a queue to produce a FIFO for the order in which we visit nodes. The overall running time is $O(n)$, for the n enqueues and n dequeues.

Algorithm breadthfirst():

Initialize queue Q to contain $\text{root}()$

while Q not empty **do**

$p = Q.dequeue()$

$\{ p \text{ is the oldest entry in the queue } \}$

perform the “visit” action for position p

for each child c in children(p) do

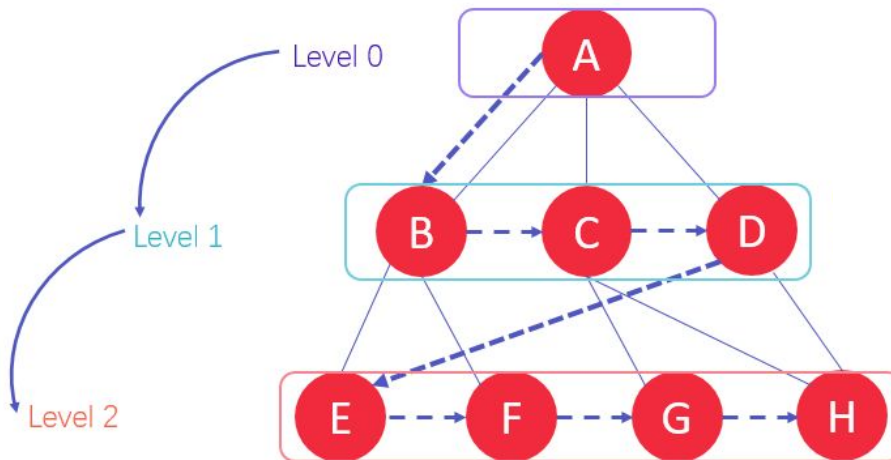
$Q.enqueue(c)$ { add p 's children to the end of the queue for later visits }

Data Structures

Traversal of a Binary Tree

Breadth-First Tree: visit all the positions at depth d before we visit the positions at depth $d + 1$.

- The algorithm traverses the graph in the smallest number of iterations and the shortest possible time.
- Due to high precision and robust implementation, BFS is used in multiple real-life solutions like P2P networks, Web Crawlers, and Network Broadcasting.



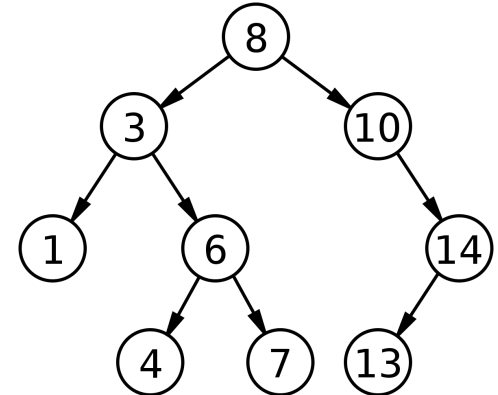
Data Structures

Binary Search Tree BST

Binary Search Tree BST

It is an algorithm for analyzing the node, its left and right branches to search, insert, and delete a value.

- Nodes of the tree are represented in a parent-child relationship
- Each parent node can have zero child nodes or a maximum of two nodes
- The right sub-tree has values **greater** than the parent node.
- The left sub-tree has values **smaller** than the parent node.



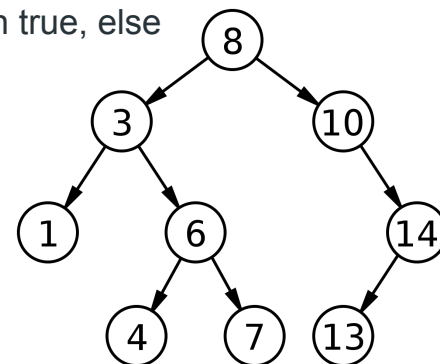
Data Structures

Binary Search Tree BST

Start analyzing tree at the root node and then move further to either the right or left subtree of the root node depending upon the element to be located is either less or greater than the root.

```
If root == NULL
    return NULL;
If number == root->data
    return root->data;
If number < root->data
    return search(root->left)
If number > root->data
    return search(root->right)
```

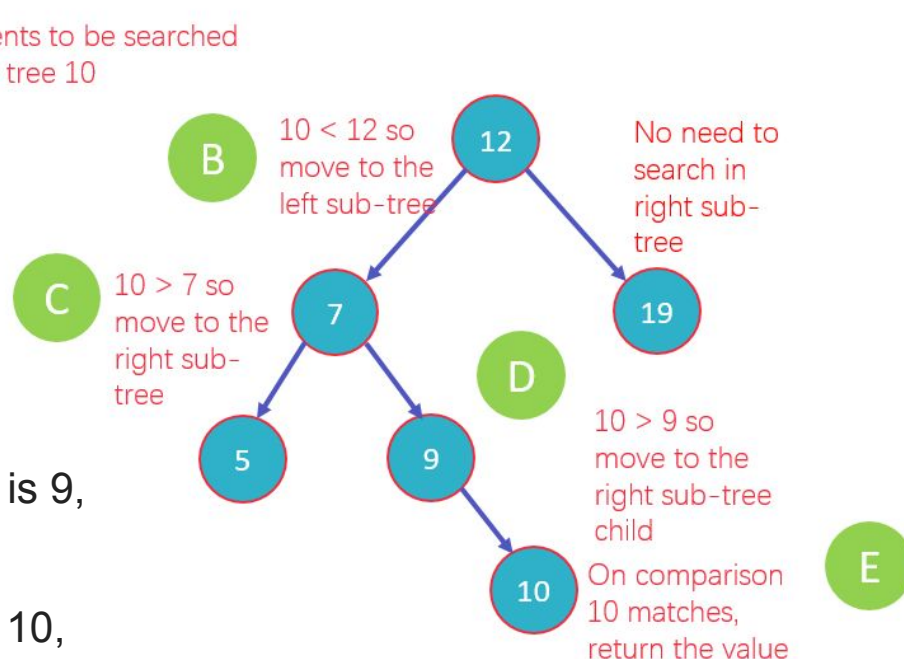
1. Start from the root.
2. Compare the searching element with root, if less than root, then recursively call left subtree, else recursively call right subtree.
3. If the element to search is found anywhere, return true, else return false.



Data Structures

Binary Search Tree BST (Example)

1. The element to be searched is 10
2. Compare the element with the root node 12, $10 < 12$, hence you move to the left subtree. No need to analyze the right-subtree
3. Now compare 10 with node 7, $10 > 7$, so move to the right-subtree
4. Then compare 10 with the next node, which is 9, $10 > 9$, look in the right subtree child
5. 10 matches with the value in the node, $10 = 10$, return the value to the user.

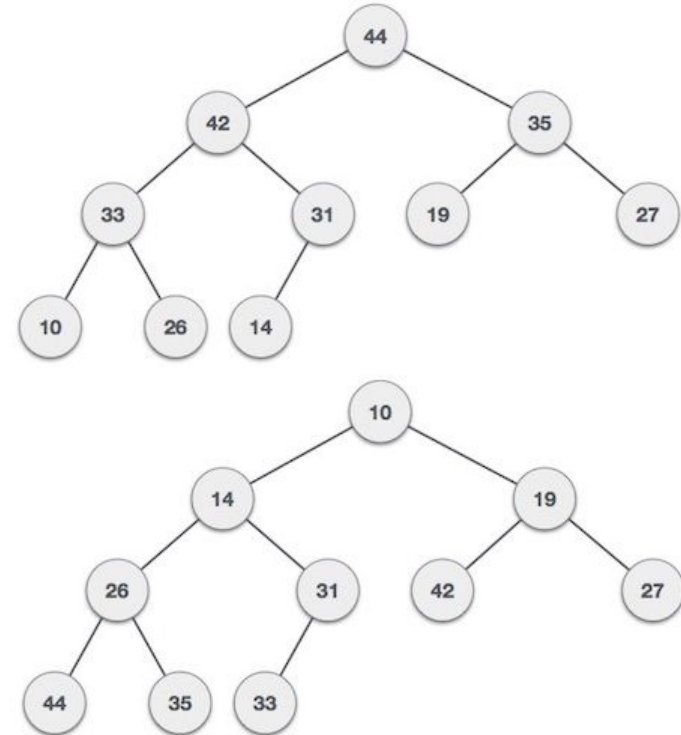


Data Structures

Heap Tree

A **Heap** is a special Tree data structure in which the tree is a complete binary tree, and of two types:

1. **Max-Heap:** In a Max-Heap the key present at the root node must be greatest among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.
2. **Min-Heap:** In a Min-Heap the key present at the root node must be minimum among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.



Data Structures

Heap Tree

Max Heap addition Algorithm

Input 35 33 42 10 14 19 27 44 26 31

- Create a new node at the end of heap.
- Assign new value to the node.
- Compare the value of this child node with its parent.
- If value of parent is less than child, then swap them.
- Repeat step 3 & 4 until Heap property holds.

